

Project Workflow

Walmart Sales Forecasting — end-to-end pipeline, from raw data to deployed dashboard

Pipeline Overview

1

Raw Data Ingestion

6,435 weekly records loaded from `Walmart_DataSet.csv` — 45 stores × 143 weeks (Feb 2010 – Oct 2012), covering sales, holidays, temperature, fuel price, CPI, and unemployment.

`pandas` `data/raw/Walmart_DataSet.csv`

2

Cleaning & Preprocessing

Dates parsed to datetime, records sorted per store, weekly frequency aligned (`W-FRI`), and each store's series verified for completeness (no missing weeks).

`pandas` `walmart_cleaned.csv`

3

Exploratory Data Analysis

Correlation analysis (Unemployment, CPI, Temperature vs. sales), seasonality by month, holiday-week sales lift, and best/worst store comparison — all computed both at the aggregate and per-store level.

`pandas` · `matplotlib` · `seaborn`

4

Model Fitting — Per Store

SARIMA(1,1,1)(1,1,1,52) fit independently for all 45 stores. Two fits per store: one on a train/test split for evaluation, one on the full history for the production forecast.

`statsmodels` (SARIMAX class)

5

Evaluation

Last 12 weeks per store held out as a test set. MAE, RMSE, and MAPE computed per store, and every store rated Excellent / Good / Needs Improvement based on its MAPE.

`scikit-learn metrics` `model_evaluation_metrics.csv` `actual_vs_predicted_holdout.csv`

6

Forecast Generation

The full-history model produces a genuine 12-week-ahead forecast per store — sales for weeks that haven't happened yet.

`forecast_next_12_weeks.csv`

7

Precompute & Cache

All of the above (steps 2–6) runs offline via `precompute.py`, not inside the live app — fitting 45 seasonal models takes several minutes, too slow to run on every dashboard page load.

`scripts/precompute.py`

8

Dashboard

A 6-page Streamlit app reads the precomputed CSVs (cached with `@st.cache_data`) and renders interactive charts, filters, and per-store drill-downs — no model fitting happens at request time.

`Streamlit` · `Plotly`

9

Deployment

Code and precomputed CSVs pushed to GitHub. Streamlit Community Cloud is connected to the repo and auto-rebuilds the app on every push to `main`.

`GitHub` · `Streamlit Community Cloud`

Refresh Cycle

When new sales data becomes available: rerun `scripts/precompute.py` (steps 2-6 above execute again, ~7-8 minutes for all 45 stores) → commit the four regenerated CSVs → push to `main` → Streamlit Cloud redeploys automatically. The dashboard code itself (`app.py`) does not need to change for a routine data refresh.

Repository Structure

Path	Role in the pipeline
<code>data/raw/</code>	Stage 1 — original untouched dataset
<code>notebooks/</code>	Stages 2-6 — the full EDA, modeling, and evaluation work
<code>scripts/precompute.py</code>	Stage 7 — regenerates all output CSVs from raw data
<code>walmart_deploy/</code>	Stage 8 — the deployed Streamlit app + its precomputed CSVs
<code>walmart_deploy/.streamlit/config.toml</code>	App theme configuration
<code>docs/</code>	Project documentation (this file and related reports)